

Aplicativo Deskotop para Gestão de Assinaturas de Jogos Educacionais**Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas****INTEGRANTES DO PROJETO e RA'S**

André Seiji Shiroma	-	26029347
Marcos Almeida Daloia	-	25027861
Fabio Augusto Camillo Rodrigues	-	26029390

São Paulo

2025

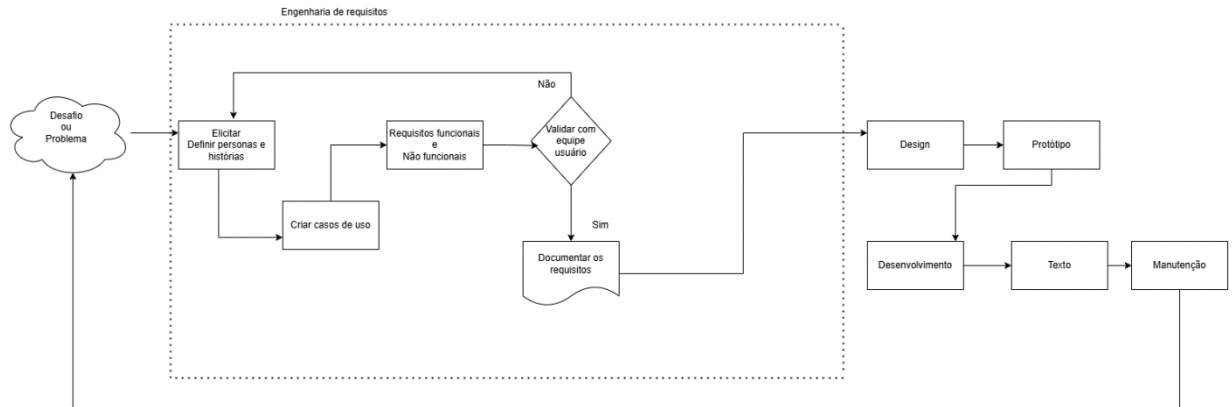
Sumário

1 INTRODUÇÃO.....	3
1.1 – Processo de desenvolvimento de Software	3
1.2 Apresentação	3
2. DESIGN SPRINT – Ideação e prototipação do desafio	3
2.1 Desafio	3
2.2 Entender Mapear	4
2.2.1 – Entendimento e observação. COMO PODEMOS?	4
2.2.2 – Pesquisa Desk	4
2.3 Ideação – desenho da solução (trilha do usuário)	4
2.4 Prototipagem	5
3.REQUISITOS DE SISTEMA	6
3.1 REQUISITOS FUNCIONAIS DE SOFTWARE	6
3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE.....	8
4. CASOS DE USO.....	12
5. DIAGRAMA DE CLASSE	Error! Bookmark not defined.
6. ARQUITETURA DO SISTEMA.....	Error! Bookmark not defined.
6.1. Visão Geral do Sistema	Error! Bookmark not defined.
6.2. Arquitetura Geral	Error! Bookmark not defined.
6.3. Componentes Arquiteturais e Camadas	Error! Bookmark not defined.
6.4. Diagrama da Arquitetura	Error! Bookmark not defined.
6.5. Organização de Pastas e Módulos	Error! Bookmark not defined.
6.6. Integrações Externas (se houver)	Error! Bookmark not defined.
6.7. Ambiente e Implantação	Error! Bookmark not defined.
8. REFERÊNCIAS BIBLIOGRÁFICAS	22

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo (substituir) - o que foi realizado em sala no Draw.io, escolha um por equipe



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote. Tendo.....

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Inicial da proposta do PI - as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

2.2 Entender Mapear

2.2.1 – Entendimento e observação. COMO PODEMOS?

- Como vamos construir de um banco de dados?
- Como o usuário vai acessar o app?
- Qual linguagem de programação iremos usar no back-end e front-end?
- Como iremos fazer o design desse app?
- Como vai ser a linguagem visual desse app?
- Como iremos fazer o instalador do app?
- Qual vai ser o framework?
- Qual vai ser a sua engine?
- Qual editor de código iremos usar?
- Como podemos deixar o software mais leve possível para rodar em qualquer máquina?
- Como podemos deixar esse app totalmente funcional offline?

2.2.2 – Pesquisa Desk

- Entender quais Frameworks são mais estáveis para a plataforma Windows.
 - Mapear como streamings por exemplo Netflix, HBO MAX ou Xbox Gamepass organiza seus catálogos.
 - Entender os jogos educacionais em tendência.
- Pesquisar sobre UX e UI.
 - Ver como é a identidade visual de cada plataforma.
 - Netflix, Disney+, Steam, Epic Games, Xbox e Playstation.
 - Como essas plataformas exibem os seus jogos? Como é a sua Vitrine?
- Pesquisar como está o mercado de jogos educacionais.
 - Ver outras concorrentes com premissas parecidas ao nosso app.
 - Quais são as mecânicas desses jogos que mais chamaram a atenção para dar destaque.

2.3 Ideação – desenho da solução (trilha do usuário)

Passo 1: O usuário irá entrar no aplicativo e cadastrar o seu RA e senha da escola como aluno.

Passo 2: Após o cadastro o usuário vai encontrar vários banners na plataforma e cada um será um jogo.

Passo 3: Ao selecionar o jogo, o usuário irá visualizar o botão play, descrição do jogo e no final da página irá conter jogos similares.

Passo 4: Ao apertar o play, a plataforma irá ser minimizada, sendo redirecionado para uma tela de loading que iniciará o jogo em alguns segundos.

Passo 5: Ao finalizar o jogo, o sistema irá mostrar um ranking com os alunos que fizeram mais pontos.

2.4 Prototipagem



TELA DE LOGIN EM
PDF.pdf



HOME PAGE EM
PDF.pdf



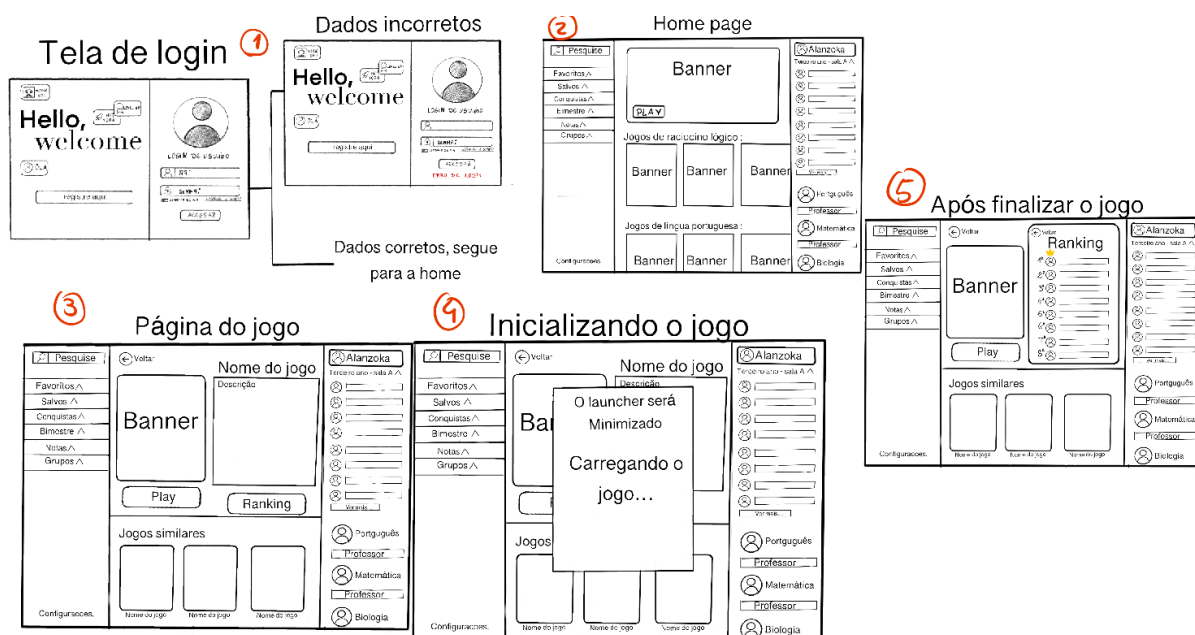
PAGINA DE JOGO
EM PDF.pdf



MINIMIZANDO
JOGO EM PDF.pdf



RANKING EM
PDF.pdf



3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Cadastro de Games
Descrição	Registrar os jogos, podendo adicionar, atualizar ou excluir.
Entradas	Título do jogo, descrição, categoria/gênero, arquivos do jogo, imagem de capa e classificação indicativa.
Fonte	Administrador do sistema ou Desenvolvedor de jogos.
Saídas	Confirmação de registro bem-sucedido, mensagem de erro (falta algum dado) e exibição do jogo na listagem da plataforma.
Ação	1. Todos os campos obrigatórios devem ser preenchidos. 2. Os arquivos enviados devem estar em formatos compatíveis. 3. O sistema deve identificar cada jogo por um ID único.

RFS02	
Função	Cadastro de Pacotes
Descrição	O sistema deve permitir ao usuário cadastrar, editar e excluir pacotes de jogos na plataforma.
Entradas	Nome do pacote, descrição, lista de jogos incluídos, preço, imagem do pacote e disponibilidade.
Fonte	Administrador do sistema.
Saídas	Mensagem de confirmação de sucesso ou mensagem de erro em caso de dados inválidos.
Ação	1. O pacote deve conter pelo menos um jogo cadastrado previamente. 2. O sistema deve associar corretamente os jogos ao pacote. 3. Cada pacote deve possuir um ID único.

RFS03

Função	Cadastro de escolas
Descrição	O sistema deve permitir ao usuário cadastrar, editar e excluir escolas na plataforma.
Entradas	Nome da escola, CNPJ, endereço, telefone, e-mail e responsável da instituição.
Fonte	Administrador do sistema
Saídas	Mensagem de confirmação de sucesso ou mensagem de erro em caso de dados inválidos.
Ação	1. O CNPJ deve ser válido e único no sistema. 2. Os campos obrigatórios devem ser preenchidos. 3. O e-mail deve estar em formato válido. 4. Cada escola deve possuir um ID único.

RFS04	
Função	Login por Escola
Descrição	O sistema deve permitir que escolas cadastradas realizem login na plataforma utilizando suas credenciais.
Entradas	CNPJ e Senha
Fonte	Instituição de Ensino
Saídas	Acesso ao sistema em caso de sucesso ou mensagem de erro em caso de falha na autenticação
Ação	1. Apenas escolas previamente cadastradas podem acessar o sistema. 2. O sistema deve validar as credenciais informadas. 3. A senha deve ser armazenada de forma segura (criptografada). 4. Após múltiplas tentativas inválidas, o sistema pode bloquear temporariamente o acesso. 5. Cada sessão deve ser identificada de forma única.

RFS05	
Função	Registro de acesso ao game
Descrição	(log com data/hora, escola, game, IP, resultado permitido/bloqueado) e incremento do contador mensal;

Entradas	Identificação da escola, identificação do jogo, endereço IP.
Fonte	Sistema
Saídas	Registro de log armazenado no sistema e atualização do contador mensal de acessos.
Ação	1. O registro deve ser feito automaticamente a cada acesso ao jogo. 2. O sistema deve registrar a data e hora automaticamente. 3. O acesso deve ser classificado como permitido ou bloqueado. 4. O contador mensal deve ser incrementado a cada acesso válido.

RFS06	
Função	Bloqueio de acesso
Descrição	Quando o limite mensal do pacote for atingido, irá aparecer uma mensagem com limite de acesso atingido
Entradas	Identificação da escola, identificação do pacote, quantidade de acessos realizados no mês.
Fonte	Sistema
Saídas	Mensagem informando que o limite de acesso foi atingido e bloqueio do acesso ao jogo.
Ação	1. O limite mensal deve ser definido conforme o pacote contratado. 2. O sistema deve verificar o limite antes de permitir o acesso ao jogo. 3. Ao atingir o limite, novos acessos devem ser bloqueados automaticamente. 4. O sistema deve informar claramente o motivo do bloqueio ao usuário. 5. O contador de acessos deve ser reiniciado a cada novo período mensal.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Modo Offline
Descrição	A aplicação deve funcionar offline (modo local) para fins de protótipo;
Entradas	Dados previamente armazenados no dispositivo (jogos, credenciais validadas e permissões de acesso)
Fonte	Sistema
Saídas	Acesso aos jogos em modo offline e sincronização dos dados quando a conexão for restabelecida.
Ação	1. O acesso offline deve ser permitido apenas para jogos previamente baixados/autorizados. 2. O usuário deve realizar login ao menos uma vez com internet antes de usar o modo offline. 3. Os dados de uso devem ser armazenados localmente enquanto offline. 4. O sistema deve sincronizar automaticamente os dados ao reconectar à internet. 5. O sistema deve impedir acesso offline a conteúdos não autorizados.

RFS02	
Função	Interface para desktop
Descrição	Interface intuitiva e compatível com desktop (Windows), com telas de lista + formulário;
Entradas	Interações do usuário via teclado e mouse.
Fonte	Usuário
Saídas	Exibição adequada das informações, formulários e navegação entre telas.
Ação	1. A interface deve ser responsiva para diferentes resoluções de tela desktop. 2. O sistema deve apresentar menus, listas e formulários de forma clara e organizada. 3. As ações devem ser acessíveis com poucos cliques. 4. O sistema deve manter padrão visual consistente entre as telas. 5. Deve ser compatível com os principais navegadores em ambiente desktop.

RFS03	
Função	Tempo de resposta
Descrição	Tempo de resposta adequado para consultas e relatórios com base de dados pequena/média;
Entradas	Requisições do usuário (cliques, envios de formulários, acesso a jogos).
Fonte	Usuário
Saídas	Respostas do sistema exibidas na interface.
Ação	1. O tempo de resposta para ações comuns (navegação e formulários) deve ser de até 2 segundos. 2. O carregamento de jogos deve ocorrer em até 5 segundos, considerando condições normais de rede. 3. O sistema deve suportar múltiplos acessos simultâneos sem degradação significativa de desempenho. 4. Em caso de lentidão, o sistema deve apresentar feedback visual (exibir mensagem de carregamento)

RFS04	
Função	Banco relacional
Descrição	Persistência em banco relacional com integridade (chaves, relacionamentos e validações);
Entradas	Dados inseridos pelo sistema (cadastros, acessos, registros).
Fonte	Sistema
Saídas	Dados armazenados e recuperados de forma consistente e estruturada.
Ação	1. O banco de dados deve garantir integridade referencial (uso de chaves primárias e estrangeiras). 2. Os dados devem ser armazenados de forma consistente, evitando redundâncias indevidas. 3. O sistema deve validar os dados antes da persistência. 4. As operações devem garantir confiabilidade (evitar perda ou corrupção de dados). 5. O modelo deve suportar os relacionamentos entre entidades (escola, jogos, pacotes, acessos).

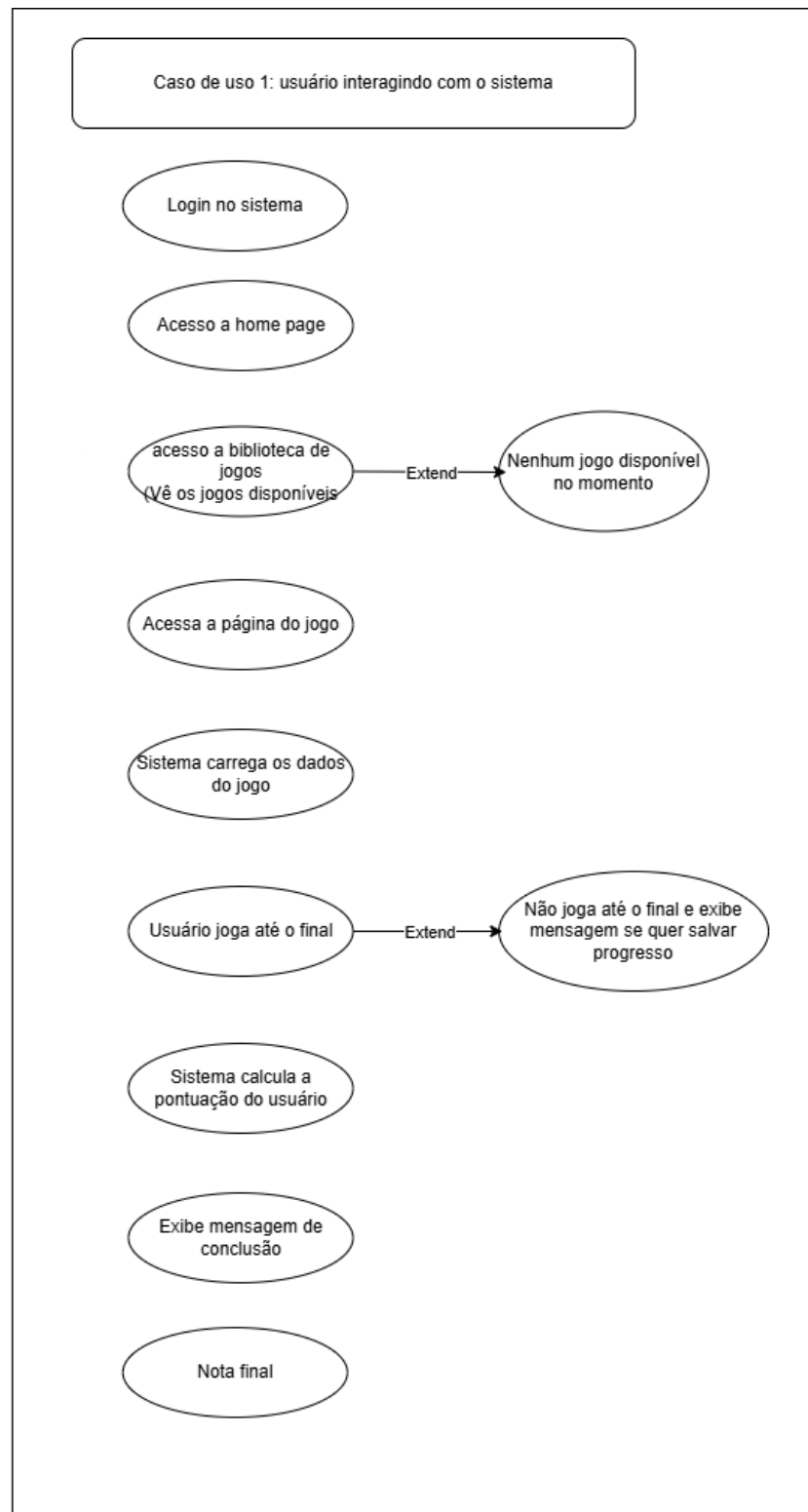
RFS05	
Função	Código Modular
Descrição	Código modular e organizado em camadas (UI, regras de negócio, dados/persistência);
Entradas	Requisições de novas funcionalidades, atualizações de sistema ou manutenção de bugs.
Fonte	Desenvolvedores
Saídas	Sistema atualizado com alterações implementadas de forma organizada e isolada, sem impactar outros módulos.
Ação	1. O código deve ser dividido em módulos independentes com responsabilidades bem definidas. 2. Alterações em um módulo não devem afetar outros módulos do sistema. 3. O sistema deve permitir reutilização de componentes. 4. A estrutura do código deve facilitar manutenção e evolução futura. 5. Deve seguir boas práticas de desenvolvimento (ex: separação de responsabilidades).

RFS06	
Função	Documentação do código
Descrição	Documentação clara e padronizada do projeto e do código (README e manual rápido de execução).
Entradas	Código-fonte desenvolvido e atualizado pela equipe.
Fonte	Desenvolvedores
Saídas	Documentação técnica atualizada, incluindo comentários no código e possíveis documentos auxiliares.
Ação	1. O código deve conter comentários explicativos nas partes mais complexas. 2. As funções, classes e métodos devem possuir descrições claras de sua finalidade. 3. A documentação deve ser mantida atualizada conforme alterações no sistema. 4. Deve seguir um padrão de documentação definido pela equipe. 5. A documentação deve permitir que outros desenvolvedores entendam o sistema com facilidade.

4. DIAGRAMAS DE UML

4.1 CASOS DE USO

Apresentar 3 casos de uso do sistema



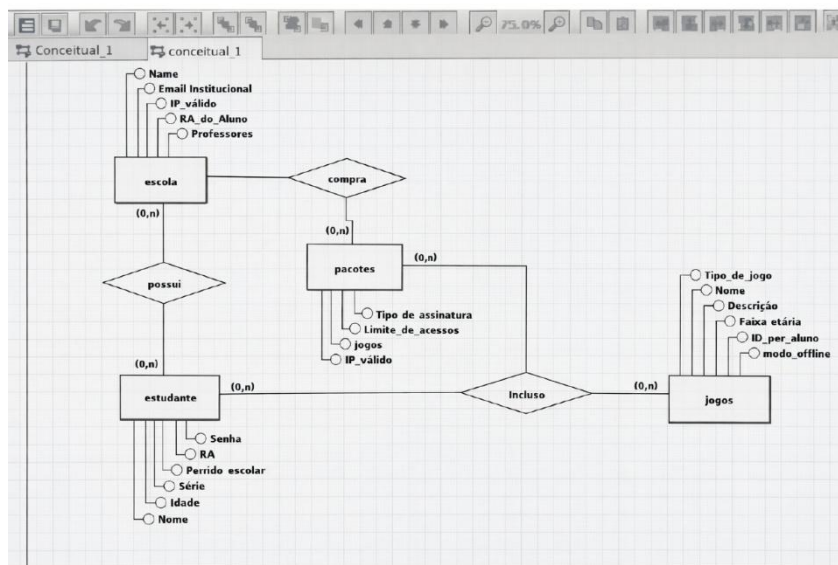


Caso de uso 3: Administrador gestão de jogos



4.2 DIAGRAMA DE CLASSE

Apresentar 1 diagrama de classe do sistema



5. Especificação estrutura de uma rotina do sistema

5.1 – Apresentar a especificação de acordo com o modelo abaixo

Sistema/Software

Módulo de Autenticação de Usuários

Função

Autenticar o aluno e validar suas credenciais de acesso.

Descrição

Verifica a identidade do aluno através do RA (Registro Acadêmico) e senha, garantindo que apenas usuários cadastrados e ativos acessem a interface de jogos.

Entradas

- RA do Aluno
- Senha criptografada
- ID da Plataforma
- Data/hora da tentativa de login

Fonte

- Banco de dados de Alunos
- Status da conta (Ativa/Inativa)
- Tabela de credenciais

Saídas

- StatusAutenticacao (Sucesso ou Falha)
- Token de Sessão (em caso de sucesso)
- Mensagem de erro (em caso de falha)
- Registro de log de auditoria

Destino

Interface Principal da Plataforma.

Ação

- Buscar o RA no banco no dados de alunos.
- Verificar se a senha fornecida corresponde com o RA armazenado
- Verificar se o status do aluno está "Ativo".
- Se credenciais inválidas ou conta inativa → bloquear acesso e exibir mensagem de erro.
- caso contrário → gerar token de sessão e permitir acesso.
- Registrar log de tentativa de acesso com:
- RA, data/hora, IP e resultado (Sucesso/Falha).

Requisitos

- As senhas devem ser armazenadas com criptografia (Hashing).
- O sistema deve bloquear a conta após X tentativas falhas (opcional/segurança).
- Conexão estável com o banco de dados de usuários.

Pré-condição

Aluno deve possuir um cadastro prévio realizado pela instituição.

Pós-condições

- Sessão do usuário iniciada.
- Log de login registrado no sistema.

Efeitos colaterais

Criação ou atualização de token de usuário.

Especificação 1 – Login do Aluno à Plataforma**Sistema/Software**

Módulo de Autenticação de Usuários

Função

Autenticar o aluno e validar suas credenciais de acesso.

Descrição

Verifica a identidade do aluno através do RA (Registro Acadêmico) e senha, garantindo que apenas usuários cadastrados e ativos acessem a interface de jogos.

Entradas

- RA do Aluno
- Senha criptografada
- ID da Plataforma
- Data/hora da tentativa de login

Fonte

- Banco de dados de Alunos
- Status da conta (Ativa/Inativa)
- Tabela de credenciais

Saídas

- StatusAutenticacao (Sucesso ou Falha)

- Token de Sessão (em caso de sucesso)
- Mensagem de erro (em caso de falha)
- Registro de log de auditoria

Destino

Interface Principal da Plataforma (Dashboard) + Base de logs de segurança

Ação

- Buscar o RA no banco de dados de alunos.
- Verificar se a senha fornecida corresponde ao hash armazenado.
- Verificar se o status do aluno está "Ativo".
 - Se credenciais inválidas ou conta inativa → bloquear acesso e exibir mensagem de erro.
 - Caso contrário → gerar token de sessão e permitir acesso.
- Registrar log de tentativa de acesso com:

RA, data/hora, IP e resultado (Sucesso/Falha).

Requisitos

- As senhas devem ser armazenadas com criptografia (Hashing).
- O sistema deve bloquear a conta após X tentativas falhas (opcional/segurança).
- Conexão estável com o banco de dados de usuários.

Pré-condição

Aluno deve possuir um cadastro prévio realizado pela instituição.

Pós-condições

- Sessão do usuário iniciada.
- Log de login registrado no sistema.

Efeitos colaterais

Criação de cookie de sessão no navegador/aplicativo e atualização da data de "Último Acesso" no perfil do aluno.

5.2 – Apresentar a codificação da rotina Apresente em pseudocódigo, Python ou outra linguagem

ALGORITMO "Login_Plataforma_Educacional"

VAR

ra_informado, senha_informada : TEXTO

ra_banco, senha_hash_banco : TEXTO

status_conta : TEXTO

tentativas : INTEIRO

autenticado : LOGICO

token_sessao : TEXTO

erro_sistema : LOGICO

INICIO

// Inicialização

autenticado <- FALSO

tentativas <- 0

erro_sistema <- FALSO

// Entradas do usuário

ESCREVA("Digite o seu RA: ")

LEIA(ra_informado)

ESCREVA("Digite sua senha: ")

LEIA(senha_informada)

// Busca dados no banco

TENTE

```
ra_banco      <- BUSCAR_RA_BANCO(ra_informado)

status_conta  <- BUSCAR_STATUS_ALUNO(ra_informado)

senha_hash_banco <- BUSCAR_HASH_SENHA(ra_informado)

CAPTURE erro_conexao

ESCREVA("Erro interno. Tente novamente mais tarde.")

REGISTRAR_LOG(ra_informado, "Falha: Erro de sistema", DATA_HORA_ATUAL())

erro_sistema <- VERDADEIRO

FIM_TENTE

// Loop de tentativas

SE (NAO erro_sistema) ENTAO

    ENQUANTO (tentativas < 3 E autenticado == FALSO) FACA    corrigido

        // Validações

        SE (ra_banco == ra_informado) ENTAO

            SE (status_conta == "Ativo") ENTAO

                SE (VERIFICAR_HASH(senha_informada, senha_hash_banco)) ENTAO

                    autenticado <- VERDADEIRO

                    token_sessao <- GERAR_TOKEN_SESSAO(ra_informado) corrigido

                    ATUALIZAR_ULTIMO_ACESSO(ra_informado)

                    REGISTRAR_LOG(ra_informado, "Sucesso", DATA_HORA_ATUAL())

                    ESCREVA("Login realizado com sucesso!")

                    ABRIR_DASHBOARD_JOGOS(token_sessao)

            SENAO

                tentativas <- tentativas + 1

                REGISTRAR_LOG(ra_informado, "Falha: Senha incorreta", DATA_HORA_ATUAL())
```

```
SE (tentativas < 3) ENTAO
    ESCRIVA("RA ou senha inválidos. Tentativa " + tentativas + " de 3.")
    FIM_SE
FIM_SE
SENAO
    autenticado <- FALSO
    ESCRIVA("Conta inativa. Procure a secretaria.")
    REGISTRAR_LOG(ra_informado, "Falha: Conta inativa", DATA_HORA_ATUAL())
    INTERROMPER // não conta como tentativa de senha
    FIM_SE
SENAO
    // RA inexistente — mesma mensagem genérica (segurança) corrigido
    tentativas <- tentativas + 1
    REGISTRAR_LOG(ra_informado, "Falha: RA inexistente", DATA_HORA_ATUAL())
    SE (tentativas < 3) ENTAO
        ESCRIVA("RA ou senha inválidos. Tentativa " + tentativas + " de 3.")
        FIM_SE
    FIM_SE
FIM_ENQUANTO
// Bloqueio após 3 tentativas
SE (tentativas >= 3 E autenticado == FALSO) ENTAO
    BLOQUEAR_CONTA_TEMPORARIA(ra_informado, 15) // 15 min
    ESCRIVA("Muitas tentativas. Conta bloqueada por 15 min.")
    REGISTRAR_LOG(ra_informado, "Bloqueio temporário", DATA_HORA_ATUAL())
    FIM_SE
FIM_SE
```

FIM_ALGORITMO

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software**. 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.

VALENTE, Marco Tulio. Engenharia de software moderna: princípios e práticas para desenvolvimento de software com produtividade. Belo Horizonte: Marco Tulio Valente, 2020. iv, 395 p. ISBN 9786500019506.

LUZ, Matheus Antonio da. Windie: jogos independentes por assinatura. 2022. 123 f. Trabalho de Conclusão de Curso - Graduação (Bacharel em Sistemas de Informação) - Câmpus Central - Sede: Anápolis - CET - Ciências Exatas e Tecnológicas Henrique Santillo, Universidade Estadual de Goiás, Anápolis, GO, 2022.

SAKUDA, Luis Ojima; FORTIM, Ivelise (Org.). **Mapeamento da Indústria Brasileira de Jogos Digitais**. São Paulo: USP/BNDES, 2014. Disponível em: https://www.abragames.org/uploads/5/6/8/0/56805537/mapeamento_da_industria_brasileira_e_global_de_jogos_digitais_1.pdf. Acesso em: 10 maio 2026.